Week 4 – Particle Filter

Orlova Valeriia 247590

April 13, 2026

1 Introduction

The goal of this assignment was to implement a localization algorithm based on a particle filter. The algorithm estimates the robot position using lidar measurements and a motion model.

1.1 Prediction

The prediction step was implemented in the function `predict_pose`, which is based on a differential drive motion model. Each particle is updated according to the wheel velocities and the sampling period. A random noise term was added to the model to represent motion uncertainty.

The influence of the noise magnitude was experimentally evaluated. With a small noise value (e.g. 0.01), the particles followed the motion model very accurately, but the filter tended to converge slowly and could remain stuck in an incorrect region.

On the other hand, with a larger noise value (e.g. 0.05), the particles spread more, which improved exploration of the state space and allowed the filter to recover from incorrect hypotheses. However, this resulted in a higher variance of the final estimate.

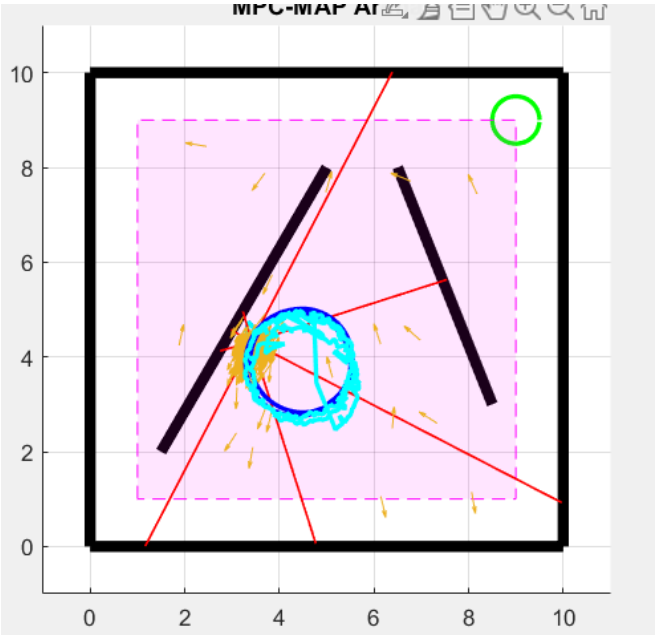

Figure 1: Small noise (0.01)

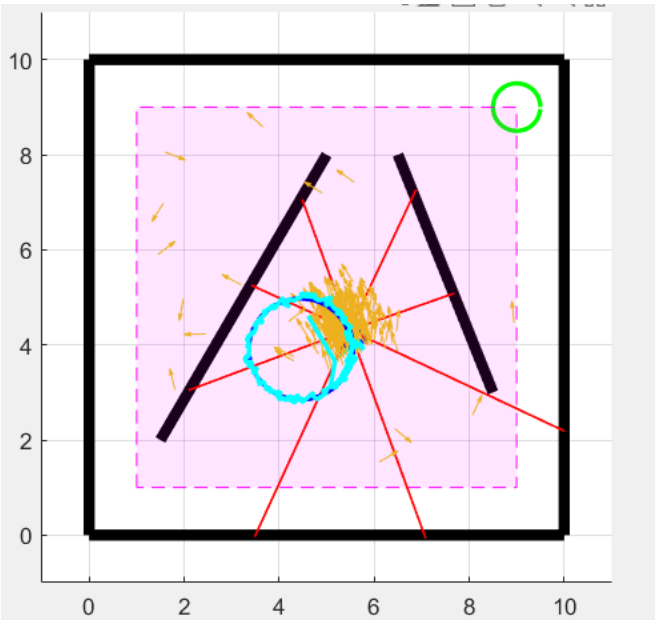

Figure 2: Larger noise (0.05)

1.2 Correction

The correction step was implemented using the function `compute_lidar_measurement`, which simulates ideal lidar measurements for each particle. The function uses `ray_cast`, provided by the simulator, to compute intersections of rays with map walls. The distance to the nearest intersection is used as the predicted measurement.

The particle weights were computed using the following metric:

$$err = \text{mean}(|z_{real} - z_{pred}|) \tag{1}$$

$$w_i = \exp(-3 \cdot err) \tag{2}$$

This approach was chosen because it is more robust to outliers than squared error. Using a quadratic error metric resulted in unstable behavior, where a few incorrect lidar rays could dominate the weight calculation.

1.3 Resampling

Resampling was implemented in the function `resample_particles` using a multinomial sampling approach based on the cumulative distribution function (CDF) of particle weights.

The effective number of particles is defined as:

$$N_{eff} = \frac{1}{\sum w_i^2} \tag{3}$$

Resampling is performed only when $N_{eff} < 0.5N$, which improves stability and reduces unnecessary fluctuations of particles. Additionally, a small fraction (approximately 5%) of randomly generated particles was introduced to prevent the filter from getting stuck in incorrect regions.

1.4 Localization

Particles were initialized randomly within the GNSS-denied region of the map. In each iteration, prediction, correction, and resampling steps were performed.

The robot was set in motion to provide informative measurements. It was observed that at the beginning of the simulation, the filter could temporarily converge to an incorrect hypothesis. However, due to the presence of noise and random particles, the filter was able to recover and converge to the correct robot position.

1.5 Conclusion

The implemented particle filter successfully localized the robot in the given environment. All main components of the algorithm, including prediction, correction, and resampling, were implemented and verified.

It was shown that parameter selection plays a crucial role in filter performance. Smaller noise values led to more precise but less robust behavior, while larger noise values improved robustness at the cost of increased variance.

The use of conditional resampling and random particle injection significantly improved stability and prevented divergence. The resulting filter was able to converge to the true robot position and track it during motion.